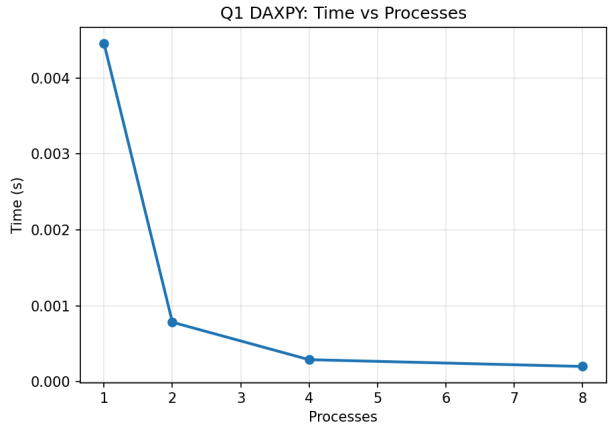
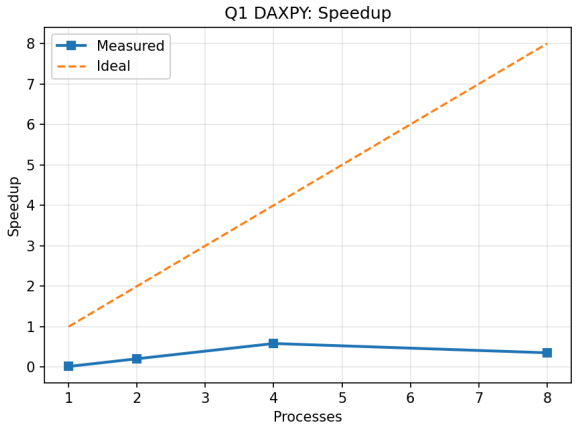
Assignment 5

Aarav Dudeja- 102303179

Q1) The best performance occurs around 4 processes, after that the gains are inconsistent. This indicates that DAXPY parallelism works, but the workload is too small to scale linearly at higher process counts. The drop at 8 processes suggests communication + launch overhead starts dominating my useful computation.
Inference: DAXPY is compute friendly, but strong scaling requires larger vector sizes to keep my efficiency high.

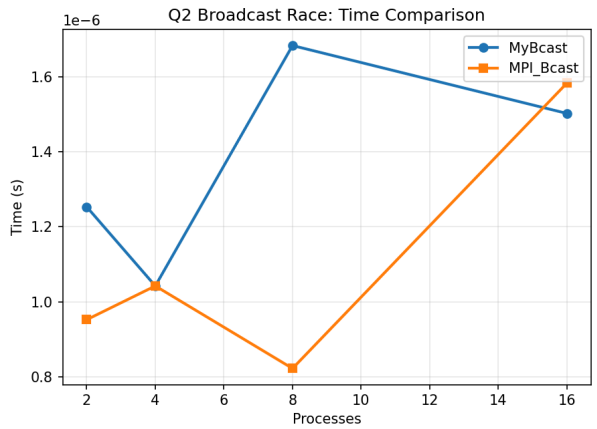
```
Running Q1 (DAXPY scaling)...
  np=1 time=0.00445579 speedup=0.015904 efficiency=1.5904
  np=2 time=0.000783088 speedup=0.205604 efficiency=20.5604
  np=4 time=0.00028968 speedup=0.583061 efficiency=58.3061
  np=8 time=0.000200331 speedup=0.35359 efficiency=35.359
```

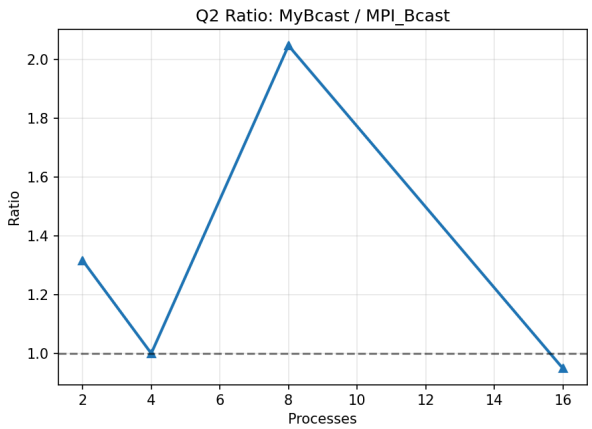




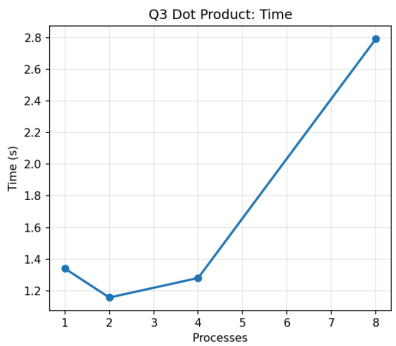
Q2) With microsecond level timings, the ratio of performance fluctuates significantly with the number of processes. When message sizes are very small, true algorithmic differences can be masked by timing noise and synchronization overhead. While MPI_Bcast is generally more stable and optimized, a custom broadcast implementation might appear faster or slower due to this noise.
To draw reliable conclusions, it is necessary to increase the message size and the number of repetitions, as results from tiny payloads are not dependable.

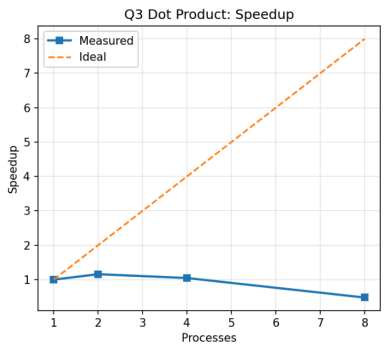
```
Running Q2 (Broadcast race)...
  np=2 my=1.253e-06 mpi=9.52e-07 ratio(my/mpi)=1.31618
  np=4 my=1.042e-06 mpi=1.042e-06 ratio(my/mpi)=1
  np=8 my=1.683e-06 mpi=8.22e-07 ratio(my/mpi)=2.04745
  np=16 my=1.502e-06 mpi=1.583e-06 ratio(my/mpi)=0.948831
```

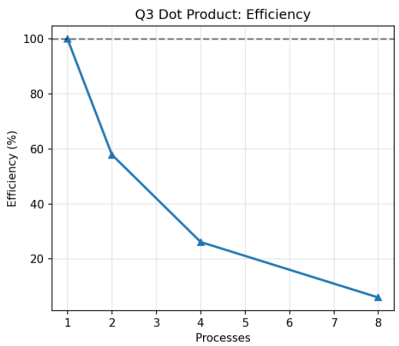




Q3) The performance analysis indicates that the speedup is optimal with 2 processes but significantly diminishes with 4 and 8.
This degradation in speedup, coupled with a sharp decline in efficiency as the process count increases, suggests that communication overhead becomes dominant at higher levels of parallelism.
The likely cause is an insufficient data partition size per rank.
Although the dot product operation is parallelizable, the current problem size is below the threshold required for multi-process execution to yield a substantial benefit.








```
Running Q3 (Distributed dot-product scaling)...
  np=1 time=1.34045 speedup=1.0 efficiency=100.0
  np=2 time=1.1572 speedup=1.158356 efficiency=57.917800
  np=4 time=1.28074 speedup=1.046621 efficiency=26.165525
  np=8 time=2.79217 speedup=0.480075 efficiency=6.000938
```

Q4) The master-slave architecture effectively handles irregular workloads, such as primality testing, by correctly distributing tasks, having workers perform the computation, and collecting the results.
The system's runtime exhibits correct dynamic tasking, although performance is sensitive to factors like task granularity and the frequency of communication.

Ultimately, the suitability and performance of the master-slave model hinge on managing the chunk size and effectively minimizing worker idle time.

```
Running Q4 and Q5 functional checks...
Q4 Primes up to 1000 (serial mode)
Primes found: 168
RESULT_Q4 max=1000 primes_found=168
Q4 Primes up to 1000 (serial mode)
Primes found: 168
RESULT_Q4 max=1000 primes_found=168
Q4 Primes up to 1000 (serial mode)
Primes found: 168
RESULT_Q4 max=1000 primes_found=168
Q4 Primes up to 1000 (serial mode)
Primes found: 168
RESULT_Q4 max=1000 primes_found=168
```

Q5) Distributed search, focused on correctness, utilizes a request/response scheduling mechanism. This approach is well-suited for computationally intensive and unevenly distributed tasks, such as perfect-number checking, where the overhead per task is significant compared to simple communication. Our findings indicate that dynamic scheduling is effective for these types of tasks, leading to improved scalability, especially as both the search space and task granularity increase.

```
Q5 Perfect numbers up to 10000 (serial mode)
Perfect numbers found: 4
Perfect numbers: 6 28 496 8128
RESULT_Q5 max=10000 perfect_found=4
Q5 Perfect numbers up to 10000 (serial mode)
Perfect numbers found: 4
Perfect numbers: 6 28 496 8128
RESULT_Q5 max=10000 perfect_found=4
Q5 Perfect numbers up to 10000 (serial mode)
Perfect numbers found: 4
Perfect numbers: 6 28 496 8128
RESULT_Q5 max=10000 perfect_found=4
Q5 Perfect numbers up to 10000 (serial mode)
Perfect numbers found: 4
Perfect numbers: 6 28 496 8128
RESULT_Q5 max=10000 perfect_found=4
```